DISTINCTION-LEVEL NUMERICAL PROJECT

Conceptual Sounding Rocket Trajectory Simulation

A detailed, week-by-week build plan for an MSc Space Engineering portfolio

Compressed 6-Week Schedule · ~12–14 hrs/week

Prepared for: A BEng Mechanical Engineering graduate building aerospace-modelling evidence for postgraduate applications.
Project: A reduced-order Python model predicting single-stage rocket flight from launch to apogee, including thrust, gravity, quadratic drag and variable mass.
Outputs: Working simulation codebase (Git) · altitude/velocity/acceleration profiles · parametric trade studies · 10–15 page technical report · 1-page portfolio summary.

00 · How to use this plan

This plan takes you from "I can write basic Python" to a clean, verified trajectory simulator with proper trade studies and a distinction-grade report. The brief's marking bands reward **correct physics, transparent modelling, clear cause-and-effect explanation, readable code, and professional figures** — not complexity. This plan is engineered to hit each of those explicitly.

THE DISTINCTION BANDS YOU'RE TARGETING

- **70–75%:** functional model, correct implementation, basic analysis.
- **75–80%:** clear structure, good explanation, consistent results.
- **80–85%:** strong analytical insight, well-presented trade studies.
- **85%+:** exceptional clarity, depth of reasoning, professional execution.

Weeks 4–6 are where you cross from "functional" into "exceptional": verification against an analytic case, a timestep-convergence study, and interpreted (not just presented) results.

RUNNING THIS ALONGSIDE THE CUBESAT PROJECT

This plan assumes ~12–14 hrs/week. The companion *1U CubeSat* plan runs over the same six weeks. They complement each other: this project is code-heavy and best done in focused blocks; the CubeSat is research/CAD. The combined calendar is in the appendix.

> **USE GIT FROM DAY ONE**
>
> A public GitHub repo with clean commits is itself portfolio evidence. Admissions tutors can click it.

· Contents

01 · Project & model definition

You will build a **1-D vertical trajectory model** first (the core required by the brief), then optionally extend to a launch angle. A fixed baseline vehicle keeps you moving — change values later, but start here.

Parameter	Symbol	Baseline
Lift-off mass	m_0	20 kg
Propellant mass	m_p	8 kg (dry mass 12 kg)
Thrust (constant)	T	1500 N
Burn time	t_b	6 s → $\dot{m}$ = 1.333 kg/s
Drag coefficient	C_d	0.5
Body diameter / area	A	0.10 m → A = 7.85×10^{-3} m²
Sea-level air density	ρ_0	1.225 kg/m³ (exponential model, H ≈ 8500 m)
Gravity	g	9.81 m/s² (constant; refine later)

These values give a burnout velocity of a few hundred m/s and an apogee on the order of several kilometres — a realistic small sounding rocket, easy to sanity-check.

> **THE FLIGHT HAS TWO PHASES — YOUR CODE MUST DISTINGUISH THEM**
>
> **Powered** ($t < t_b$): thrust on, mass decreasing. **Coast** ($t \geq t_b$): thrust off, constant mass, decelerating under gravity + drag until velocity reaches zero = **apogee**. The brief explicitly asks you to separate these.

02 · Tools, setup & repository structure

STACK (ALL FREE)

- **Python 3** with `numpy` (arrays/math) and `matplotlib` (figures).
- **Jupyter** for exploration, but ship clean `.py` modules for the final code.
- **Git + GitHub** for version control and portfolio visibility.
- **Overleaf (LaTeX)** or Word for the report.

REPOSITORY LAYOUT

```
# clean structure earns "readable code" marks
rocket-sim/
  src/
    model.py      # physics + RK4
    config.py     # vehicle params
    run.py        # main driver
    plots.py      # figure generation
  figures/
  report/
  README.md
  requirements.txt
```

03 · Sequenced reading list

Week	Read this	So you can…

1	NASA Beginner's Guide to Rockets — forces, thrust, weight, drag	Get the free-body diagram right before coding.
2	NASA guide — thrust & Newton's 2nd law; *Automate the Boring Stuff* (basics) if needed	Code the powered vertical model.
3	Anderson, *Introduction to Flight* — drag equation, dynamic pressure	Add quadratic drag + exponential atmosphere.
4	Sutton, *Rocket Propulsion Elements* — rocket equation (concept level); RK4 method	Add variable mass; upgrade integrator; verify.
5	Light reading on performance trade-offs & centre of mass/ pressure (stability extension)	Design and interpret the trade studies.
6	Review notes only — no new material	Write and polish.

04 · The 6-week schedule

Build incrementally — a working-but-simple model every week beats a broken complex one. Total ≈ 72–84 hrs.

Week 1 — Foundations & free-body diagram

~11 hrs

Get the physics and the tooling right before any trajectory code.

- Refresh Python: variables, functions, `numpy` arrays, `matplotlib` plotting. Produce one throw-away motion graph (e.g. projectile under gravity) to confirm your environment works.
- Read the NASA guide; **draw the free-body diagram** for both flight phases (thrust, weight, drag).
- Initialise the Git repo and the folder structure above; write a one-line `README`.
- Write down the governing equation of motion in your own words (see §05).

Deliverable: Working Python environment + one motion plot + free-body diagram + initialised repo.

Week 2 — Powered vertical model (thrust vs gravity)

~13 hrs

First real trajectory: constant mass, no drag — get the loop right.

- Implement a **time-stepping loop** (Euler first — simplest) updating acceleration → velocity → altitude.
- Model thrust during burn and gravity throughout; ignore drag and mass change for now.
- Plot **altitude vs time** and **velocity vs time**.
- Sanity-check: does it accelerate up under thrust, then decelerate under gravity? Does apogee occur where v = 0?

Deliverable: Altitude-vs-time graph from a working powered model (Euler).

Week 3 — Aerodynamic drag

~14 hrs

Make it realistic: add quadratic drag and a varying atmosphere.

- Add the **drag force** $D = \frac{1}{2}\rho v|v|C_d A$ (use v|v| so drag always opposes motion).
- Add an **exponential atmosphere** $\rho(h) = \rho_0 \cdot e^{-h/H}$ so density falls with altitude.
- Compare apogee **with vs without** drag — quantify how much altitude drag costs. This is your first cause-and-effect result.
- Plot velocity and altitude; note the terminal-velocity behaviour on descent if you extend past apogee.

Deliverable: Velocity + altitude plots with drag; a sentence quantifying drag's effect on apogee.

Week 4 — Variable mass, RK4 & verification

~15 hrs

This week earns the distinction: correct mass loss, a better integrator, and proof it's right.

- Implement **variable mass**: $m(t) = m_0 - \dot{m}\cdot t$ during burn, then constant. Use the correct system-level equation (§05.3) — do **not** double-count the exhaust momentum term.
- Upgrade the integrator from Euler to **RK4** for accuracy and stability.
- **Verify** against the analytic no-drag case (Tsiolkovsky burnout velocity minus gravity loss; coast height = $v^2/2g$). Agreement builds credibility.
- Run a **timestep convergence study** (§06): show apogee converges as Δt shrinks. This single figure separates 80%+ work from the rest.

Deliverable: Full powered+coast trajectory (altitude, velocity, acceleration vs time) + verification note + convergence plot.

Week 5 — Parametric trade studies

~13 hrs

Investigate how design variables drive performance — isolate one at a time.

- Sweep each variable independently and plot **apogee vs variable**: thrust, propellant mass, burn time, C_d, cross-sectional area.
- Hold everything else fixed; vary one parameter across a sensible range (e.g. ±50%).
- For each, write the **physical explanation** of the trend (e.g. "higher C_d lowers apogee because drag scales with C_d, costing kinetic energy during ascent").
- Optional extension (extra marks): non-vertical launch angle, or a simple stability note (centre of mass vs centre of pressure).

Deliverable: 4–5 trade-study figures, each with an explained trend.

Week 6 — Report, figures & polish

~14 hrs

Turn correct results into a professional, interpreted document.

- Write the **report** (§08 structure): methodology, assumptions, results, trade studies, limitations.
- Remake every figure to **publication quality**: labelled axes with units, legends, titles, consistent style, readable fonts.
- Clean the code: functions, docstrings, a `README` with run instructions and a results figure; tidy the Git history.
- Write the **assumptions & limitations** section critically (constant g, no rotation, constant C_d, 1-D, exponential atmosphere).
- Produce the **1-page portfolio summary** (§10).

Deliverable: Final 10–15 page report + clean codebase (GitHub) + labelled figure set + 1-page summary.

05 · Physics & numerics cheat-sheet

05.1 Governing equation (Newton's 2nd law)

For vertical flight, with upward positive, the net force divided by instantaneous mass gives acceleration:

$$m(t) \cdot dv/dt = T(t) - m(t)\cdot g - D(v,h) \qquad dh/dt = v$$

During **coast**, T = 0 and m is constant. Solve this system numerically in time.

05.2 Aerodynamic drag (quadratic)

$$D = \tfrac{1}{2} \cdot \rho(h) \cdot v\cdot|v| \cdot C_d \cdot A \qquad \rho(h) = \rho_0 \cdot e^{-h/H}$$

The `v·|v|` form makes drag oppose motion in both ascent and descent. H ≈ 8500 m is the atmospheric scale height. State the exponential atmosphere as an explicit assumption.

05.3 Variable mass — the equation to get right

Mass falls linearly while burning, then holds:

$$m(t) = m_0 - \dot{m}\cdot t \; (t < t_b) \; , \; \dot{m} = m_p / t_b$$

THE CLASSIC MISTAKE TO AVOID

The measured **thrust T already includes** the exhaust-momentum term ($\dot{m}\cdot v_e$). In the system-level form `m·dv/dt = T − mg − D`, you must **not add a separate $\dot{m}\cdot v_e$ term** — that double-counts the propulsion. Use the effective exhaust velocity only if you instead define thrust *from* it ($T = \dot{m}\cdot v_e$). Pick one formulation and state it.

05.4 Time integration — Euler then RK4

Euler (first order, simple, for Week 2):

$$v_{n+1} = v_n + \Delta t \cdot a(v_n,h_n,t_n) \qquad h_{n+1} = h_n + \Delta t \cdot v_n$$

RK4 (fourth order, far more accurate for the same Δt — your final integrator). Sketch implementation:

```
def deriv(state, t):
    h, v = state
    m = mass(t)
    thrust = T if t < t_b else 0.0
    drag = 0.5*rho(h)*v*abs(v)*Cd*A
    a = (thrust - m*g - drag)/m
    return np.array([v, a])

def rk4_step(state, t, dt):
    k1 = deriv(state, t)
    k2 = deriv(state + 0.5*dt*k1, t + 0.5*dt)
    k3 = deriv(state + 0.5*dt*k2, t + 0.5*dt)
```

```
    k4 = deriv(state + dt*k3,     t + dt)
    return state + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)
```

05.5 Apogee detection

Apogee is where velocity crosses zero on the way down. Detect the sign change between steps and linearly interpolate the apogee time/altitude for a clean value rather than the nearest grid point.

05.6 Useful analytic checks (for verification)

```
Ideal Δv (no drag/gravity) = v_e · ln(m₀ / m_dry) [Tsiolkovsky] Gravity loss ≈ g · t_b    Coast height (no drag) = v_bo² / (2g)
```

With $v_e = T/\dot{m} \approx 1125$ m/s and $\ln(20/12) \approx 0.511$, ideal $\Delta v \approx 575$ m/s; subtract gravity loss (~59 m/s) to estimate burnout velocity, then coast height. Your full simulator (no-drag mode) should reproduce these — a powerful credibility check.

06 · Verification & the timestep convergence study

The brief explicitly rewards "logical and transparent modelling" and "correct use of governing equations." Two cheap pieces of evidence prove both:

1. **Analytic verification.** Run the simulator with drag switched off and compare burnout velocity and coast apogee to the hand calculations in §05.6. Report the percentage agreement.
2. **Timestep convergence.** Run the full model at Δt = 0.1, 0.05, 0.02, 0.01, 0.005 s and plot apogee vs Δt. Show it converges to a stable value, then justify your chosen Δt as a balance of accuracy and runtime.

> **WHY THIS MATTERS FOR THE GRADE**
>
> Most students present one trajectory and stop. Showing that your numbers are *independent of the numerical choices* is precisely the "depth of reasoning" that pushes a report from the 75% band into 85%+.

07 · Trade-study framework

Isolate one variable, sweep it, plot apogee (and optionally burnout velocity), then *explain the physics of the trend*.

Variable swept	Expected effect on apogee	Physical reason to state
Thrust T	Increases	Higher acceleration → higher burnout velocity → more coast height (diminishing as drag grows with v^2).
Propellant mass m_p	Increases (with diminishing returns)	More Δv via the log term, but heavier vehicle and longer gravity loss.
Burn time t_b (fixed impulse)	Non-trivial optimum	Short, high-thrust burns cut gravity loss but raise peak drag; a trade-off worth highlighting.
Drag coefficient C_d	Decreases	Drag scales linearly with C_d, bleeding kinetic energy during ascent.
Cross-section A	Decreases	Same mechanism as C_d; motivates slender designs.

08 · Code structure & report mapping

CODE QUALITY CHECKLIST

- Parameters in one `config.py` — no magic numbers buried in loops.
- Pure functions for forces and derivatives; the integrator separate from the physics.
- Docstrings + comments explaining *why*, not just what.
- A single `run.py` that reproduces every figure.
- `README` with how-to-run, assumptions, and a headline result image.

REPORT SECTIONS → MARKING BANDS

- **1. Introduction & objectives** — scope, vehicle.
- **2. Methodology** — equations, integrator, phases.
- **3. Assumptions & limitations** — stated & critiqued.
- **4. Verification** — analytic check + convergence.
- **5. Results** — altitude/velocity/accel profiles, apogee.
- **6. Trade studies** — interpreted, not just plotted.
- **7. Discussion & conclusion** — cause-and-effect.

Aim for 10–15 pages. Put the full code in an appendix or link the repo. Every figure: labelled axes, units, caption, and a reference in the text.

09 · Distinction tips, pitfalls & combined calendar

DO

- Build incrementally; commit working versions often.
- Interpret every figure — say *why* the curve bends.
- Verify against analytic cases and quote the agreement.
- State each assumption and its likely impact on accuracy.
- Keep one consistent parameter set across all results.

AVOID

- Double-counting thrust and exhaust momentum (§05.3).
- Linear instead of quadratic drag; forgetting `v·|v|` sign.
- One huge timestep — unstable Euler results.
- Presenting plots with no axes labels/units or no explanation.
- Adding complexity (multi-stage, 6-DOF) before the basics are verified.

COMBINED PARALLEL CALENDAR (BOTH PROJECTS)

Week	Rocket focus (≈12–14 hr)	CubeSat focus (≈12–15 hr)
1	Python refresh + free-body diagram + repo	Mission, ConOps, constraints
2	Powered vertical model (Euler)	Requirements, block diagram, orbit calc
3	Add quadratic drag + atmosphere	CAD + power budget
4	Variable mass + RK4 + verification	Battery, thermal, ADCS
5	Trade studies + figures	Comms, payload, link budget
6	Report + clean code + summary	Trade-offs, integration, write-up

10 · Personal-statement framing

A MODEL PARAGRAPH YOU CAN ADAPT (DON'T COPY VERBATIM)

"I built a reduced-order rocket trajectory simulator in Python to model single-stage flight from launch to apogee, incorporating thrust, gravity, quadratic aerodynamic drag and variable mass during propellant burn. I implemented an RK4 integrator, separated the powered and coast phases, and verified the model against an analytic no-drag solution and a timestep-convergence study before running parametric trade studies on thrust, drag and propellant mass. Interpreting those results taught me to connect numerical behaviour back to the underlying physics, and to be explicit about the assumptions — constant gravity, a 1-D model, constant drag coefficient — that bound the model's validity."

ONE-PAGE PORTFOLIO SUMMARY — INCLUDE

- A headline altitude-vs-time plot and one trade-study figure (visual hook).
- Apogee, burnout velocity and max acceleration as headline numbers.
- One line on verification (analytic agreement + convergence).
- A link to the GitHub repository.

- Two sentences on next steps (variable g, 2-D launch angle, ISA atmosphere) — shows self-critique.

FINAL NOTE

The brief prizes clarity, correctness and completeness over complexity. A clean, verified, well-explained model with interpreted trade studies is a distinction submission — every week above is built to produce exactly that.